

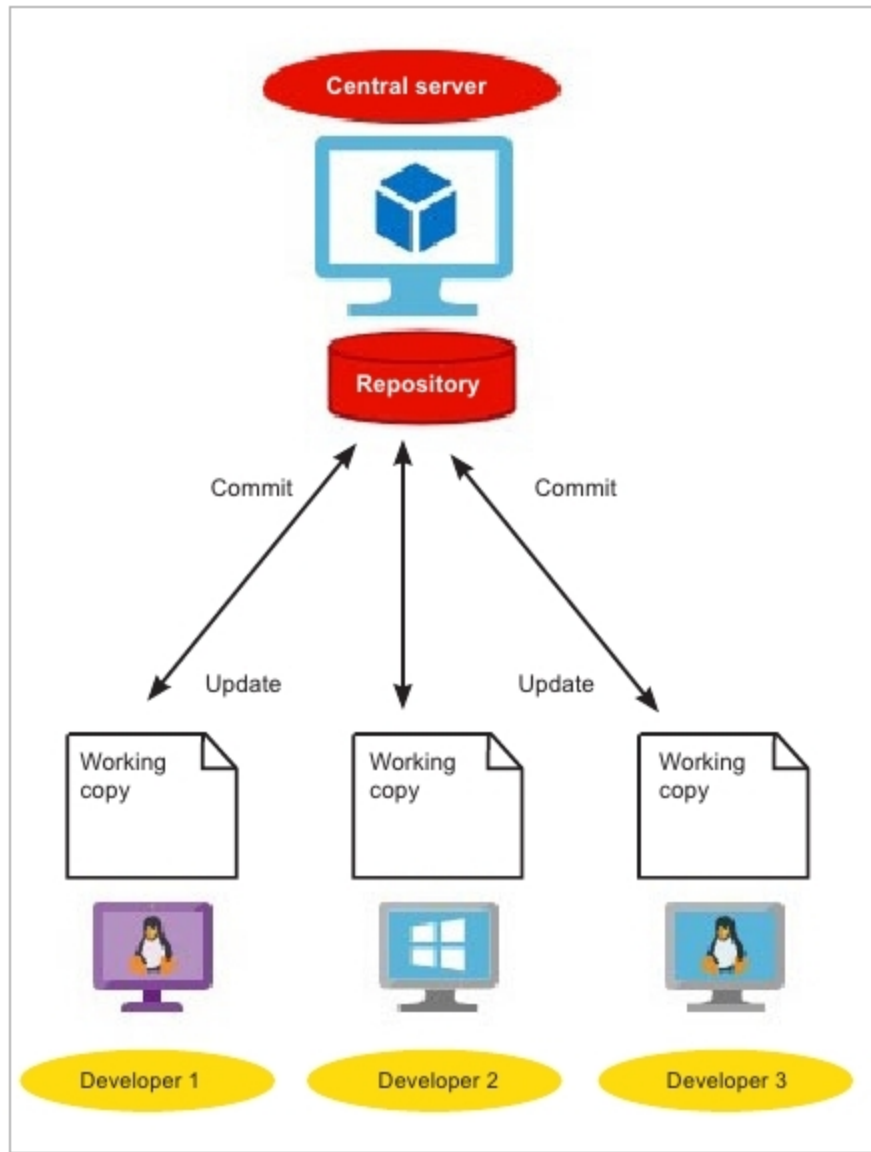


Figure 1: Centralised version control system

Non-linear and parallel development: One of the problems Git developers understood was that development is never a linear process. There are always some parallel changes that continue to happen. Hence, strong support for non-linear and parallel development was crucial. Git provides this by supporting rapid branching and merging, and by offering tools to visualise and traverse this non-linear development history. A branch in Git is extremely lightweight as it is a reference to just one commit. With the help of parent commits, the full branch structure can be constructed.

Snapshot based vs delta based version control: One major difference between Git and the traditional VCSs is the way they handle data. Most traditional VCSs store the initial file in its entirety and then only store the subsequent changes made to it. These changes are stored in files called deltas. This is the delta-based approach to

version control. Because Git was inspired by the Linux file system, it stores data as a series of snapshots. So every time a commit is made or a project saved, Git takes a snapshot of how the files looked at that moment. If the file doesn't change, then instead of storing the file, a reference to the earlier file is stored. This functionality makes Git almost like a mini file system rather than a simple VCS.

Local operations: Because Git is a distributed version control system, it gives developers a local directory to work with. Each Git directory is a full-fledged repository that contains all the versions and the entire development history. There is no need for any network connection or a central server.

Garbage accumulation and collection: Many operations like untidy commits or abort operations may leave dangling objects in the Git database. Git accumulates all the garbage, and once it is big enough it automatically collects all of it.

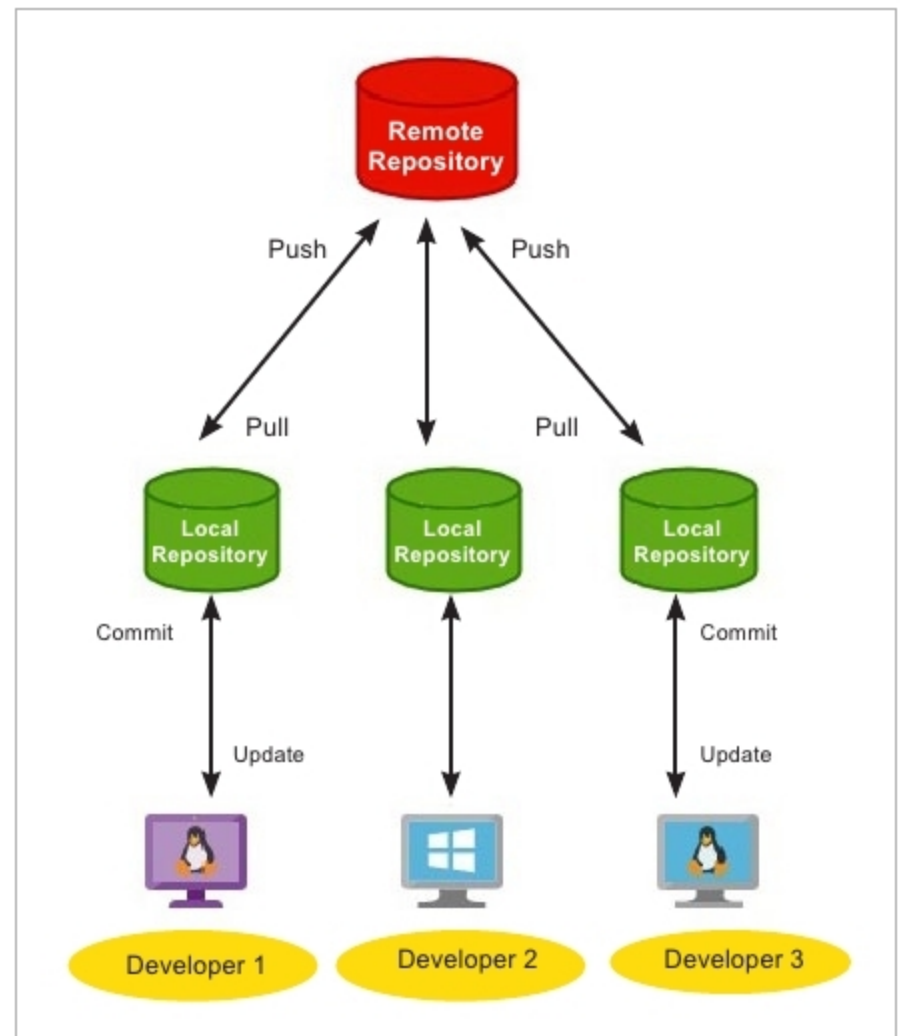


Figure 2: Distributed version control system

Garbage collection can also be explicitly done using the following command:

```
$git gc --prune.
```

Periodic object packing in packfiles: Git creates a new binary object for every new version of a file. It stores every object as an individual file. This approach takes up a lot of space and is inefficient. So to solve this problem, Git compresses a lot of objects into a single file called a *packfile*. An index file is also created for every *packfile* which stores the offset for each of the objects. This packing of objects is done at regular intervals. Since this operation of packing is computationally expensive, Git schedules it at a time when computational requirements are low.

The above characteristics make Git the most commonly used VCS. Figure 3 compares different VCSs with Git (<https://www.openhub.net/repositories/compare>).